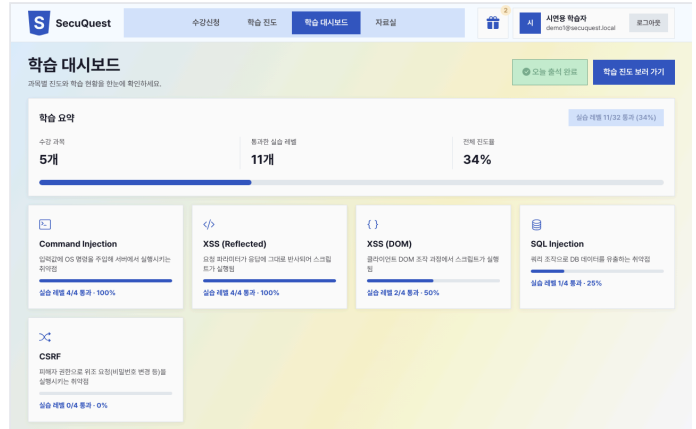


# SecuQuest

공격해 보고, 막는 법까지 배우는 웹 보안 실습

8개 웹 취약점을 하 → 중 → 상 → 안전 4단계로 직접 실습하며, 공격이 통하는 코드와 막히는 코드의 차이를 체감하도록 만든 **한국어 학습 플랫폼**입니다.

- 개발 오현명 · 1인 개발 (기획 · 설계 · 프론트엔드 · 백엔드 · 인프라)
- 기간 2026.08 ~ 2026.09.30 · 부트캠프 캡스톤 프로젝트
- 스택 Vue 3 · Flask 3 · MariaDB · Redis · Nginx · Docker Compose
- GitHub [github.com/gusaud9424-source/Final-Project](https://github.com/gusaud9424-source/Final-Project)



|                                   |                     |                       |                     |                            |                            |
|-----------------------------------|---------------------|-----------------------|---------------------|----------------------------|----------------------------|
| <b>8 × 4</b><br>취약점 과목 ×<br>실습 레벨 | <b>400</b><br>퀴즈 문항 | <b>48</b><br>REST API | <b>10</b><br>DB 테이블 | <b>6</b><br>Docker<br>컨테이너 | <b>79</b><br>설계 · 작업<br>문서 |
|-----------------------------------|---------------------|-----------------------|---------------------|----------------------------|----------------------------|

## OVERVIEW

### 왜 만들었나

|                                                                                 |                                                                                         |                                                                                                   |
|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>문제</b><br>취약점을 이론으로만 배우면 "왜 막아야 하는지"가 와닿지 않고, 실제로 공격해 볼 안전한 한국어 실습 환경은 드뭅니다. | <b>해결</b><br>같은 취약점을 방어 수준이 다른 4개 레벨로 나눠, 공격이 성공하는 코드부터 완전히 막히는 코드까지 순서대로 직접 시도하게 했습니다. | <b>안전</b><br>위험한 실습 코드는 네트워크가 끊긴 격리 컨테이너에서만 실행하고, 서비스 자체는 CSP · CSRF · Rate Limit · 감사 로그로 보호합니다. |
|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|



## KEY FEATURES

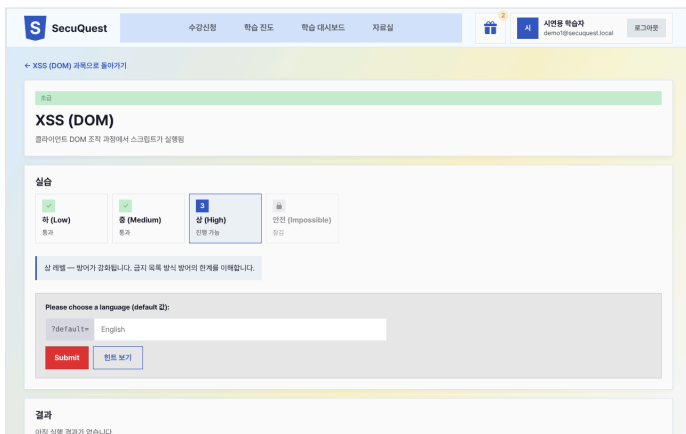
# 핵심 기능

학습자가 막힘없이 "개념 → 실습 → 방어 확인"으로 이어지도록, 그리고 운영자가 이탈 지점을 볼 수 있도록 설계했습니다.

## 01 · 실습

### 4단계 레벨 실습과 순차 잠금

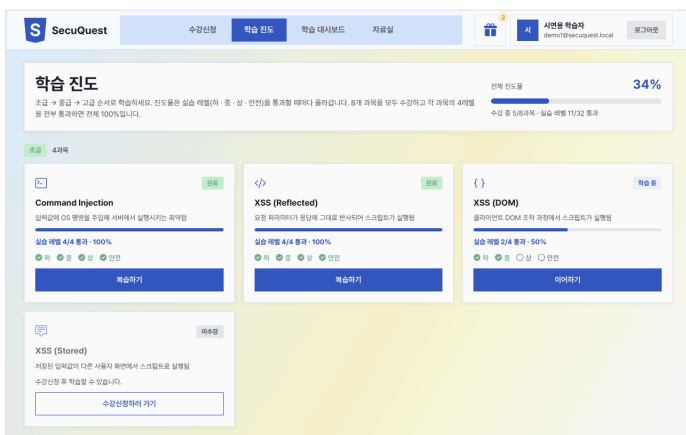
- XSS 3종 · Command Injection · SQL Injection(일반 · Blind) · File Upload · CSRF, **8과목 × 4레벨**
- 이전 레벨을 통과해야 다음 레벨이 열림 — **화면 + 서버 (403) 이중 검사**
- 안전 레벨**에서는 같은 공격이 막히는 것을 확인하며 방어 원리를 학습
- 실습 코드는 **격리 Sandbox 컨테이너**, XSS 는 전용 CSP + sandbox iframe 에서만 실행



## 03 · 동기 부여

### 레벨 기준 진도율과 보상 시스템

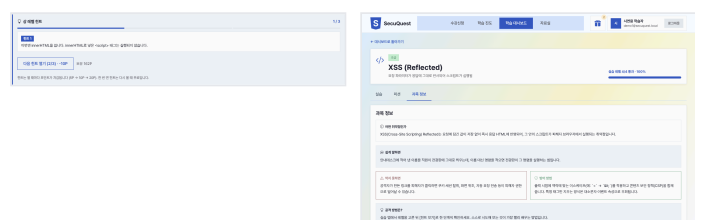
- 진도율 = **통과한 실습 레벨 ÷ 32** — 실제로 실습한 만큼만 오르도록 기준을 명확히 함
- 경험치 · 레벨 · 포인트, 레벨 클리어 보물상자, 미션 보상, 14일 출석 체크판
- 과목별 하 · 중 · 상 · 안전 통과 현황을 한 화면에서 확인



## 02 · 학습 지원

### 단계별 힌트 · 개념 확인 · 퀴즈

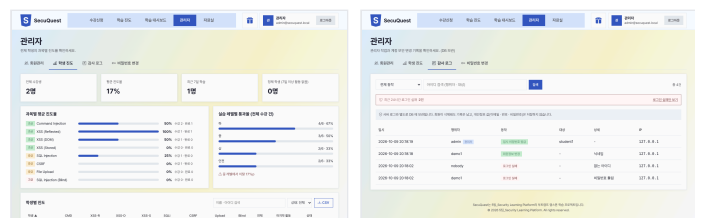
- 힌트는 한 번에 하나씩 열리고 **5P → 10P → 20P** 포인트 차감 — 정답을 바로 보지 않도록 유도
- 개념 확인 문제는 **맞음 / 틀림만 공개**(정답 비공개)해 찍어서 넘어가지 않게 함
- 과목별 3회차 퀴즈(문항 은행 400개), 자료실 학습 문서 내려받기



## 04 · 운영

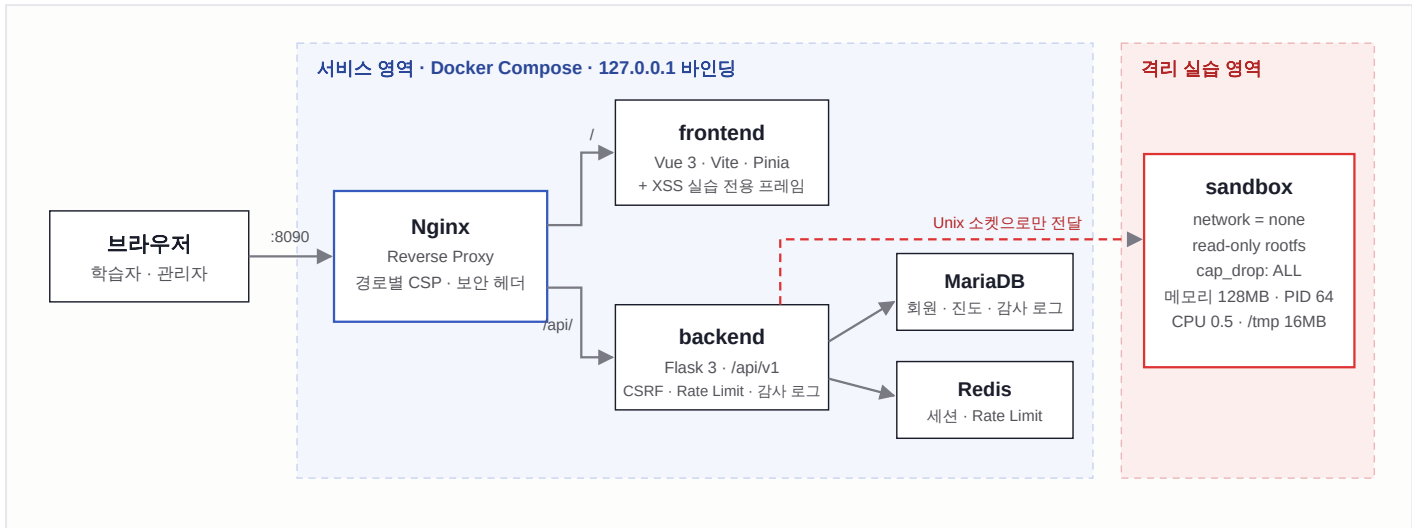
### 관리자 대시보드 · 감사 로그

- 과목별 평균 진도율, **레벨별 통과율로 이탈 지점 파악**, 전체 학생 표시, CSV 내보내기
- 회원 검색 · 임시 비밀번호 발급 · 삭제
- 관리자 작업 · 정보 변경 · **로그인 실패(24시간 집계)**를 감사 로그로 조회



# 시스템 구조와 보안 설계

"위험한 실습은 격리된 곳에서만, 서비스는 표준 보안으로 안전하게" — 두 영역을 물리적으로 분리했습니다.



## ■ 위험한 것은 격리

- **Sandbox 컨테이너:** 네트워크 없음 · 읽기 전용 · 권한 전부 제거 · 자원 제한 — 7개 항목 실측 점검
- 실습 코드는 공유 볼륨의 **Unix 소켓**으로만 전달, 서비스 DB 에 접근 불가
- XSS 실습은 **별도 경로 + 실습 전용 CSP**(외부 통신 차단) + sandbox iframe

## ■ 서비스는 표준 보안

- **인증:** Redis 세션, 로그인 시 세션 ID 재발급, bcrypt, HttpOnly · SameSite 쿠키
- **요청 위조 · 남용:** CSRF 토큰 자동 첨부, 로그인 분당 10회 · 인증코드 분당 5회 제한
- **헤더:** 앱 script-src 'self', API default-src 'none', 클릭재킹 · MIME 스니핑 방어
- **계정 보호:** 가입 여부 비노출, 인증코드 5분 만료 · 5회 오답 무효, 감사 로그

## 문제 해결 사례

각 사례는 원인 → 해결 → 검증을 저장소 docs/ 에 문서로 남겼습니다.

### CASE 01 · 보안 vs 기능

#### 엄격한 CSP 를 켜자 XSS 실습이 멈춤

원인 srcdoc iframe 은 부모 페이지의 CSP 를 그대로 상속 → 실습용 스크립트까지 차단

해결 실습 전용 페이지로 분리해 **그 경로에만** 실습용 CSP(인라인 허용 · 외부 통신 차단) 적용, postMessage 로 내용 전달

앱 전체는 script-src 'self' 유지 + 실습은 정상 동작

### CASE 02 · 정보 노출

#### 응답 시간 차이로 가입 여부가 드러남

원인 없는 아이디는 bcrypt 비교를 건너뛰어 응답이 눈에 띄게 빠름

해결 없는 아이디도 **더미 해시로 bcrypt 비교**해 응답 시간 통일, 실패 사유는 감사 로그에만 구분 기록

화면 응답은 동일 · 관리자는 24시간 실패 집계로 이상 징후 확인

### CASE 03 · 데이터 무결성

#### 감사 로그가 컨테이너와 함께 사라짐

원인 감사 기록이 서버 로그 파일에만 남아 컨테이너 재생성 시 유실

해결 audit\_logs 테이블에 **변경과 같은 트랜잭션**으로 기록 (변경 성공 ↔ 기록 성공), 개인정보 값은 미기록

회원 삭제 후에도 기록 보존 · 관리자 화면에서 검색

### CASE 04 · 학습 설계

#### 레벨을 열기만 해도 "통과"

원인 안전 레벨은 공격이 항상 막히므로, 페이지 조회만으로 통과 조건이 충족됨

해결 **직접 값을 넣어 시도한 요청만** 통과로 인정하도록 서버 판정 수정, 통과 즉시 화면 갱신

8과목 × 4레벨 = 32레벨 전체 수동 통과 검증

### CASE 05 · 동시성

#### 힌트 포인트 차감의 중복 · 경쟁 상태

원인 버튼을 빠르게 두 번 누르거나 재수강하면 같은 힌트가 두 번 차감될 수 있음

해결 포인트 원장에 힌트별 **UNIQUE 참조키** + 차감 시 **행 잠금(SELECT ... FOR UPDATE)**, 열린 힌트만 응답

재수강 후에도 산 힌트 유지 · 중복 차감 0건

### CASE 06 · 지표 설계

#### 진도율 100% 의 의미가 모호함

원인 미션 · 퀴즈 · 개념 문제가 섞여 "실습을 다 했는지"를 진도율로 알 수 없음

해결 진도율을 **실습 레벨 통과 수 ÷ 전체 32레벨**로 재정의, 학습자 · 관리자 화면과 이탈 분석을 같은 기준으로 통일

관리자가 "어느 레벨에서 멈추는지"를 바로 확인

# 기술 스택과 담당 범위

1인 개발로 기획부터 배포 환경 구성, 점검까지 전 과정을 진행했습니다.

|                                                                                                              |                                                                                                           |                                                                             |                                                                                     |
|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>Frontend</b><br>Vue 3 (Composition API) · Vite · Vue Router · Pinia · Axios · Bootstrap 5 (SCSS) · 디자인 토큰 | <b>Backend</b><br>Python 3.12 · Flask 3 · SQLAlchemy · Alembic · Flask-WTF(CSRF) · Flask-Limiter · bcrypt | <b>Data</b><br>MariaDB 11.4 (테이블 10 · 마이그레이션 6) · Redis 7 (세션 · Rate Limit) | <b>Infra</b><br>Docker Compose · Nginx 1.27 · 격리 Sandbox 컨테이너 · Gmail SMTP · SMS 연동 |
|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------|

|                 |                                                                                        |
|-----------------|----------------------------------------------------------------------------------------|
| <b>기획 · 설계</b>  | 학습 흐름(개념 → 4단계 실습 → 방어 확인) 설계, 화면 · API · ERD 설계, 진도율 · 보상 규칙 정의                       |
| <b>프론트엔드</b>    | 화면 14개 · 컴포넌트 구현, 디자인 토큰 기반 UI, 실습 프레임 · 힌트 패널, 휴대폰 · 태블릿 반응형                          |
| <b>백엔드</b>      | REST API 48개, 인증 · 회원 · 아이디/비밀번호 찾기(이메일 · SMS), 실습 판정 · 레벨 잠금, 보상 · 출석, 관리자 · 감사 로그    |
| <b>보안 · 인프라</b> | Sandbox 격리 설계 · 실측 점검, Nginx 경로별 CSP · 보안 헤더, CSRF · Rate Limit, Docker Compose 원클릭 실행 |
| <b>품질 · 문서</b>  | 전 기능 E2E 점검(브라우저 자동화), 32레벨 수동 통과 검증, 작업 기록 · 개념 설명 문서 79개, 5분 시연 시나리오                 |

|                                                                                                                                                                                                                                                                     |                                                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>배운 점</b> <ul style="list-style-type: none"><li>보안과 기능이 충돌할 때 <b>보호 범위를 경로 단위로 나누는 설계</b>가 둘 다 지키는 방법이라는 것</li><li>"동작한다"와 "의도대로 판정한다"는 다르다 — 통과 조건 · 차감 같은 규칙은 <b>서버에서, 경계 상황까지</b> 검증해야 한다는 것</li><li>문제를 원인 → 해결 → 검증으로 문서화하니 같은 실수를 반복하지 않고 설명도 쉬워짐</li></ul> | <b>다음 개선 계획</b> <ul style="list-style-type: none"><li>HTTPS 적용과 운영 환경 분리 (현재는 로컬 학습용으로 127.0.0.1에만 공개)</li><li>과목별 1회차 전용 개념 확인 문항 추가</li><li>핵심 API 자동 테스트 · CI 구성으로 회귀 점검 자동화</li></ul> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

SecuQuest · 오현명 · 2026

실습 페이지는 학습을 위해 의도적으로 취약하게 만들어져 로컬에서만 실행합니다 · [GitHub](https://github.com/gusaud9424-source/Final-Project) (https://github.com/gusaud9424-source/Final-Project)